

Arquitectura propia frente a Bedrock AgentCore

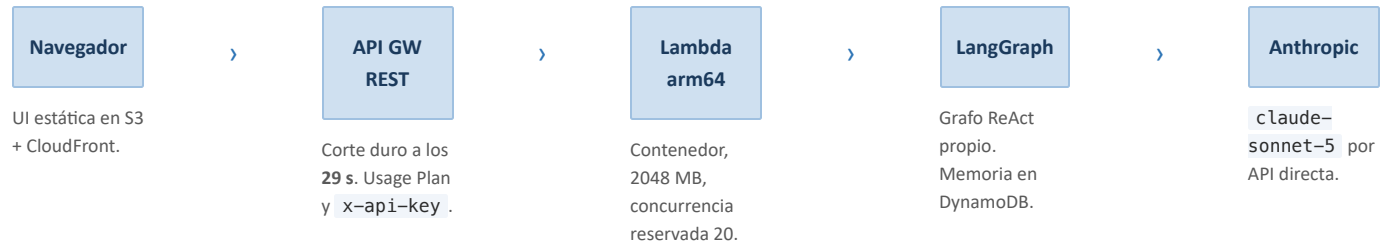
Qué cambia, qué no cambia y dónde está exactamente la línea que separa a una de la otra.

29 s	0,40 USD	7
TECHO DE TURNO DE LA ARQUITECTURA ACTUAL	INFRAESTRUCTURA AWS RECURRENTES AL MES	PRIMITIVAS GESTIONADAS QUE OFRECE AGENTCORE

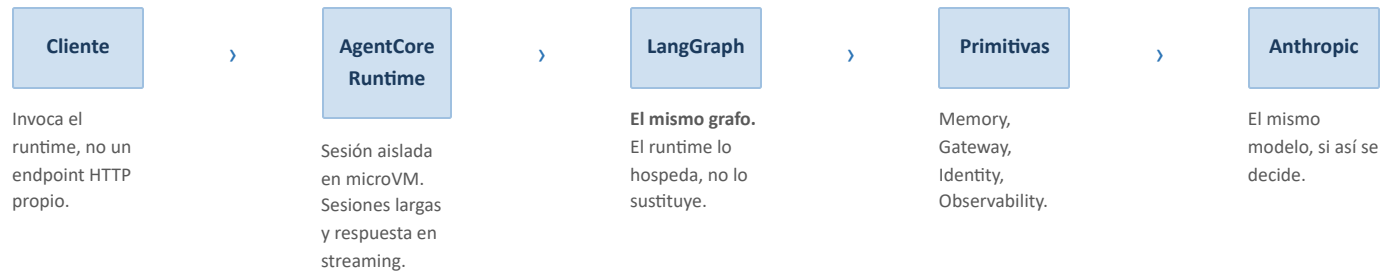
1

Qué se está comparando exactamente

La pregunta «¿usamos AgentCore?» esconde una comparación que hay que enunciar con precisión antes de responderla, porque formulada mal no tiene respuesta. No se comparan dos frameworks de agentes ni dos modelos de lenguaje. Se comparan dos **sustratos de ejecución**: el sitio donde vive el bucle del agente y las piezas que ese sitio te da ya hechas.



Arquitectura actual, según §2.1 del PRD. Bedrock aparece solo como proveedor de embeddings Titan V2, fuera de este camino.



2 Tres confusiones que conviene despejar antes

- **AgentCore no sustituye a LangGraph.** El runtime es agnóstico de framework: hospeda LangGraph, CrewAI, LlamaIndex, Strands o un bucle escrito a mano. Migrar no significaría reescribir `src/agent/graph.py` ; significaría cambiar dónde se ejecuta.
- **AgentCore no obliga a usar modelos de Bedrock.** Es agnóstico de modelo. La llamada a la API de Anthropic con `claude-sonnet-5` puede conservarse tal cual, con su clave en SSM. Esto importa mucho más de lo que parece, y la sección 5 explica por qué.
- **AgentCore no es Bedrock Agents con otro nombre.** Son dos productos con filosofías opuestas. En *Bedrock Agents* declaras *action groups* y AWS orquesta por ti: pierdes el control del bucle. En *AgentCore* traes tu propia orquestación y AWS pone el runtime y las primitivas. Para un PRD que especifica el grafo nodo a nodo (§5.1) y el límite de rondas al detalle (§5.2), la diferencia es decisiva: Bedrock Agents sería incompatible con el diseño; AgentCore no.

3 Comparación dimensión a dimensión

Dimensión	Arquitectura actual	Con AgentCore
Sustrato de ejecución	Lambda de contenedor <code>arm64</code> , 2048 MB	AgentCore Runtime, sesión aislada en microVM
Duración máxima de un turno Diferencia mayor	29 s, corte duro de API Gateway REST	Horas, orientado a sesiones largas
Respuesta en streaming	Descartada explícitamente al conservar REST (§0.3)	Soportada de forma nativa
Framework del agente	LangGraph, elegido por el PRD	LangGraph, sin cambios
Modelo de razonamiento	<code>claude-sonnet-5</code> por API de Anthropic	El mismo, o cualquier otro
Memoria conversacional	DynamoDB a mano: PK/SK, TTL, ventana de 8 mensajes (§4.5)	AgentCore Memory: corto y largo plazo gestionados
Registro de herramientas	Lista Python en <code>src/tools/__init__.py</code>	AgentCore Gateway expone APIs y Lambdas como herramientas MCP
Autenticación	<code>x-api-key</code> + Usage Plan (D-03)	AgentCore Identity, con OAuth y acceso delegado
Observabilidad	Powertools, EMF y LangSmith, montados a mano (§11)	Trazas OpenTelemetry integradas de serie
Techo de gasto	Estructural: cuota de 2.000 peticiones al mes (§2.3)	Por consumo; el techo hay que construirlo aparte

Dimensión	Arquitectura actual	Con AgentCore
Coste de infraestructura	≈ 0,40 USD/mes , medido (§9.3)	Por vCPU y memoria consumidas; sin medir en este caso
Superficie de código propio	Alta: memoria, límites, identidad y observabilidad se escriben	Menor: cuatro de esas piezas vienen dadas
Portabilidad fuera de AWS	Alta: LangGraph en un contenedor corre en cualquier sitio	Menor: el runtime y las primitivas son de AWS

La columna izquierda está verificada contra el PRD v2.7 y es citable. La derecha procede de conocimiento de producto y debe contrastarse con la documentación vigente de AWS: ver sección 8.

4

Las tres diferencias que de verdad cambian el diseño

De la tabla anterior, la mayoría de filas son matices de implementación. Tres no lo son: cambian qué producto se puede construir, no cómo se construye.

4.1 El muro de los 29 segundos

El techo de 29 s no es un parámetro del PRD: es el **eje sobre el que gira todo el diseño**. API Gateway REST corta la integración a los 29 s con independencia del `timeout` que se configure en la Lambda, y de ahí se derivan, en cascada, media docena de decisiones del documento.

Decisión del PRD	Sección	Existe por el muro de 29 s
<code>timeout</code> de la Lambda fijado a 29 s	§2.2	Un valor mayor solo desplaza quién emite el error
Presupuesto de latencia repartido en cinco tramos	§2.2	Hay que caber, y se contabiliza tramo a tramo
<code>MAX_TOOL_ROUNDS = 3</code>	§5.2	Cuatro llamadas al modelo como máximo por consulta
Ping de calentamiento cada 5 minutos	§2.2	El arranque en frío es el único riesgo real de superar los 29 s
Streaming de tokens descartado	§0.3	Consecuencia directa de conservar REST
Índice RAG obligado a pesar menos de 20 MB	§6.3	La descarga tiene que caber en el arranque

Seis decisiones que no son independientes entre sí: todas cuelgan de la misma restricción. AgentCore Runtime la levanta.

Ahora bien, aquí está el punto que hace que la respuesta sea la contraria de la que se esperaría. **Este producto no necesita más de 29 segundos**. El usuario es un agente de mostrador con un pasajero delante, y el PRD fija como métrica de éxito una latencia $p95 \leq 8\text{ s}$ (§1.3). Es un requisito de producto, no una limitación tolerada. Un agente que pudiera pensar durante ocho horas no serviría de nada aquí: nadie va a esperar.

Dicho de otro modo: AgentCore levanta un muro contra el que este caso de uso **no está chocando**. Y la restricción que sí duele — cuatro llamadas al modelo como máximo — tampoco desaparecería, porque no viene del muro sino del presupuesto: cada ronda extra cuesta dinero (§9.2, palanca 4).

4.2 Memoria gestionada frente a memoria con lógica de negocio dentro

A primera vista, AgentCore Memory sustituye limpiamente a la tabla `aeronova-memory` y ahorra el trabajo de §4.5: la clave compuesta, el relleno de ceros en la *sort key*, el TTL en segundos, la ventana de ocho mensajes. Todo eso es fontanería, y delegarla es atractivo.

El problema es que en esa tabla no solo hay memoria. Hay **tres controles que el PRD colocó ahí deliberadamente**, y que una memoria gestionada te daría que reconstruir por fuera:

- **El cortacircuitos de coste por sesión.** El ítem `STATE` acumula `cost_usd_acumulado` y, superados 0,25 USD, la sesión se rechaza con 429 (§12A.4). Es lo único que impide que una sola sesión se coma el presupuesto del mes, porque la cuota mensual protege el total pero no el reparto.
- **La comprobación de propiedad de sesión.** Si el `employee_id` no coincide con el dueño de la sesión, 403 (§4.5). Sin ella, cualquier portador de la API key lee la conversación de otro empleado: fue el hallazgo 13 del PMO.
- **La separación entre estado y mensajes.** El `pnr_activo` vive en `STATE`, no en el historial. Por eso el truncado por presupuesto L-4 puede descartar mensajes antiguos *sin* que el agente olvide el PNR, y por eso la interfaz puede prometerle al usuario que «los datos de la reserva se conservan» (§10.2). Es una decisión de diseño, no un detalle de almacenamiento.

Una memoria gestionada te da memoria. No te da tu cortacircuitos de coste. La migración sería más barata en fontanería y más cara en lógica de la que no se habla hasta que falta.

4.3 Coste: previsible por construcción frente a proporcional al uso

Este es el punto donde la comparación se vuelve incómoda, y conviene decirlo sin adornos.

La arquitectura actual gasta **0,40 USD al mes** de infraestructura AWS, y ese número no es una estimación optimista: sale del desglose de §9.3, partida por partida, para las 2.000 consultas del techo mensual. Lambda escala a cero, DynamoDB es `PAY_PER_REQUEST`, CloudFront cae en el nivel gratuito. Con cero tráfico, el coste es cero.

Más importante todavía: el techo de gasto es **estructural**. La cuota de 2.000 peticiones al mes del Usage Plan (§2.3) no depende de que nadie vigile nada, y esa propiedad se ganó a pulso: el hallazgo 38 del PMO detectó que con `period = DAY` el mismo diseño permitía ~525 USD/mes, **26 veces el techo acordado**.

AgentCore Runtime factura por consumo de vCPU y memoria por segundo. A este volumen —2.000 consultas al mes, turnos de segundos— es perfectamente posible que salga barato, o incluso más barato. Pero **«posible» no es el estándar de este proyecto**: el PRD entero está construido sobre números verificados, con un margen deliberado entre el gasto previsto (9,20 USD) y el techo duro (17,90 USD). Sustituir el sustrato invalida §9.3 y §9.4 completas y obliga a medir de nuevo, no a suponer.

Y hay una pérdida cualitativa que no se recupera midiendo: el techo estructural desaparece. Un límite de gasto por consumo hay que construirlo con presupuestos y alarmas, que *avisan*, mientras que una cuota *impide*. No es la misma clase de control.

Conviene dimensionar el alcance real de una migración, porque es bastante menor de lo que sugiere la palabra. Toda la mitad de datos del PRD queda intacta.

DATOS Y GOBIERNO — SIN CAMBIOS

- Arquitectura medallion Bronze / Silver / Gold
- Data contracts Pydantic y expectativas E-01 a E-09
- Cuarentena y manifiestos de linaje
- Índice LanceDB en S3 con puntero CURRENT
- Embeddings Titan V2 por Bedrock
- Rollback del índice en segundos

AGENTE Y CALIDAD — SIN CAMBIOS

- Grafo LangGraph y AgentState
- Las tres herramientas y su sobre ToolResult
- Defensas D-1 a D-6 contra inyección
- Límites L-1 a L-6 de entrada
- Golden dataset: 13 familias y 9 umbrales
- **Caché de prompt con TTL de 1 hora**

El último punto merece detenerse. La caché de prompt es **la principal palanca de coste del sistema**: un 25,2 % de ahorro, elegido con criterio —el TTL se ajusta al hueco entre peticiones, ~20 minutos, no a la duración de la conversación (§5.3)—. Es una capacidad de la API de Anthropic, no del sustrato. Mientras la llamada al modelo siga saliendo hacia Anthropic, **la caché sobrevive a la migración**. Solo se perdería si la migración aprovechara para pasar los modelos a Bedrock, que es una decisión distinta y separable.

6 Qué habría que rehacer del PRD

Sección	Qué contiene	Impacto
§2.1 – §2.3	Topología, parámetros de Lambda, API Gateway y Usage Plan	Reescritura
§9.3 – §9.5	Modelo de coste, presupuesto y guardarraíles	Reescritura
§4.5	Esquema de memoria, TTL y propiedad de sesión	Reescritura
§13	Runbook de despliegue en dos stacks de Terraform	Reescritura
§2.5	Permisos IAM del rol de ejecución	Revisión
§11	Observabilidad, métricas EMF y alarmas	Revisión
§12A.4	Cortacircuitos de coste por sesión	Revisión
§8.4	Prueba de carga y su modo dry_run	Revisión
§5, §6, §6A, §7, §8.2	Grafo, RAG, medallion, datos sintéticos y golden dataset	Sin cambios

Cuatro secciones a reescribir y cuatro a revisar, sobre dieciséis. El corazón del PRD —cómo razona el agente y cómo se gobiernan los datos— no se toca.

7 Cuándo cambiaría la respuesta

La conclusión de este documento no es «AgentCore no sirve», sino que **resuelve problemas que este producto no tiene**. Eso puede dejar de ser cierto, y conviene saber reconocer cuándo. Estos son los disparadores concretos:

- **Un turno deja de caber en 29 segundos.** Si aparece una consulta que exige encadenar seis o siete herramientas, o esperar a un sistema externo lento, el muro pasa de teórico a real y AgentCore se vuelve la respuesta natural.
- **Se pide streaming de tokens.** Está descartado hoy por conservar REST (\$0.3). Si la percepción de velocidad se vuelve un requisito, el sustrato actual no lo da.
- **El agente pasa a escribir, no solo a consultar.** Emitir o modificar reservas rompe el radio de explosión acotado de D-3, y el PRD ya avisa de que exige rehacer el análisis de amenazas (R-20, impacto crítico). En ese escenario, la identidad delegada y los permisos por agente de AgentCore Identity dejan de ser un lujo.
- **Aparecen varios agentes que deben coordinarse.** Un solo agente con tres herramientas no necesita infraestructura de agentes; media docena colaborando, sí.
- **Las herramientas se multiplican o se comparten.** Con tres, un registro en Python sobra. Con treinta, o con herramientas que otros equipos deban reutilizar, un Gateway con MCP deja de ser burocracia y empieza a ahorrar trabajo.
- **El volumen crece dos órdenes de magnitud.** A 2.000 consultas al mes, Lambda escalando a cero es imbatible. A 200.000, la comparación de costes hay que rehacerla entera y podría invertirse.

Ninguno de los seis se cumple hoy. Y hay una asimetría que conviene tener presente: **la arquitectura actual es la más fácil de abandonar**. Un grafo LangGraph dentro de un contenedor corre en Lambda, en AgentCore Runtime, en Fargate o en una máquina bajo la mesa. Empezar por el sustrato más simple y portable no cierra ninguna puerta; empezar por el gestionado sí acopla.

8

Qué está verificado y qué debes comprobar tú

Un documento de estudio que no distingue sus fuentes vale poco, así que aquí va la separación explícita.

Verificado contra documents/PRD.md v2.7 — citable	
Afirmación	Dónde
El PRD no menciona AgentCore ni Bedrock Agents en ninguna de sus 1.578 líneas	grep : 0 resultados
Bedrock se usa solo para embeddings Titan V2	13 menciones, todas de embeddings
Único permiso de Bedrock: InvokeModel sobre el ARN del modelo	\$2.5
Corte duro de 29 s y sus seis decisiones derivadas	\$2.2, \$5.2, \$6.3, \$0.3
Infraestructura AWS recurrente ≈ 0,40 USD/mes	\$9.3
Techo estructural por cuota mensual; con DAY serían ~525 USD/mes	\$2.3, hallazgo 38
Caché de prompt con TTL de 1 h: ahorro del 25,2 %	\$5.3, \$9.3
Latencia p95 ≤ 8 s como métrica de éxito	\$1.3

- **Contrasta con la documentación vigente de AWS** todo lo relativo a AgentCore: la lista de primitivas, la duración máxima de sesión, los límites de *payload*, la disponibilidad por región y, muy especialmente, las **tarifas**. Este documento describe el *modelo* de precios —consumo de vCPU y memoria por segundo— y deliberadamente **no cita ninguna cifra**, porque una comparación de costes con números recordados sería peor que no tener comparación.
- **Si algún día se evalúa en serio**, el criterio de decisión debería ser el mismo que usa el PRD en el resto de sus comparaciones (§9.2): coste mensual medido frente a la alternativa, no idoneidad conceptual. Fue lo que descartó OpenSearch Serverless y Streamlit en Fargate, y es una disciplina que conviene no abandonar justo en la decisión más grande.

La arquitectura actual y AgentCore no se diferencian en el framework ni en el modelo —ambos serían idénticos—, sino en **quién sostiene la sesión y quién pone las piezas de agente**; y para un asistente de mostrador que debe responder en ocho segundos, gastar menos de medio dólar al mes y no poder escribir nada, el sustrato simple y portable es el que encaja, mientras que las ventajas del gestionado —sesiones largas, streaming, identidad delegada, coordinación multiagente— resuelven problemas que este producto todavía no tiene.